

■ Strings

A **string** is a sequence of characters enclosed in quotes. Strings are **immutable** — once created, they can't be changed directly.

```
str1 = "hello world"
str2 = 'Prime'
```

Useful String Operations

Operation	Syntax	Example / Result
Length	<code>len(s)</code>	<code>len('Prime') → 5</code>
Concatenation	<code>s1 + s2</code>	<code>'Apna' + ' College' → 'Apna College'</code>
Loop	<code>for ch in s:</code>	iterates character by character
Index access	<code>s[i]</code>	<code>s[0]→'P' s[-1]→last char</code>
Slice	<code>s[start:stop:step]</code>	<code>s[::-1]</code> reverses the string

Indexing — Zero-Based!

```
s = "Python"
print(s[0]) # 'P'
print(s[3]) # 'h'
print(s[-1]) # 'n' (last char with negative index)
```

Slicing → `string[start : stop : step]`

```
s = "Python"
print(s[0:2]) # 'Py'
print(s[2:]) # 'thon'
print(s[:3]) # 'Pyt'
print(s[::2]) # 'Pto' (every 2nd char)
print(s[::-1]) # 'nohtyP' (reversed!)
```

■ *start is inclusive, stop is exclusive, step defaults to 1.*

String Formatting

Two modern ways to build dynamic strings:

Method	How it works
<code>.format()</code>	Use <code>{}</code> as placeholders → <code>"Hi {}".format(name)</code>
f-strings ■	Prefix with <code>f</code> → <code>f"Hi {name}"</code> — preferred way!

🌸 Python Fundamentals 🌸 Part 3

Strings • Lists • Tuples • Dictionaries • Sets

```
# .format()
name, age = "Rahul", 25
text = "My name is {} and I am {} years old".format(name, age)

# f-strings (Python 3.6+) ← use this!
text = f"My name is {name} and I am {age} years old"

# Expressions inside f-strings
a, b = 5, 10
print(f"sum = {a + b}") # sum = 15
print(f"avg = {(a+b)/2}") # avg = 7.5
```

■ *Named & positional placeholders: '{n}'.format(n='Bob') or '{1},{0}'.format('x','y')*

■ Lists

A **list** is an **ordered**, **mutable** collection that allows duplicates and mixed data types. Written with square brackets `[]`.

```
my_list = [1, 2, 3, 4, 5]
my_list2 = [10, "Hello", 3.14, True, 10] # heterogeneous & duplicates OK
```

List Characteristics

#	Property	Meaning
1	Ordered	Items have a fixed position, accessible by index
2	Mutable	Items can be changed after creation
3	Allows duplicates	Same value can appear more than once
4	Heterogeneous	Can hold different data types in one list

Indexing, Modifying & Slicing

🌸 Python Fundamentals 🌸 Part 3

Strings • Lists • Tuples • Dictionaries • Sets

```
# Indexing (zero-based)
lst = ["apple", "banana", "cherry"]
print(lst[0]) # apple
print(lst[-1]) # cherry

# Modifying
lst2 = [1, 2, 3, 4]
lst2[0] = 10
print(lst2) # [10, 2, 3, 4]

# Slicing → list[start:stop:step]
nums = [0,1,2,3,4,5,6,7,8,9]
print(nums[2:5]) # [2, 3, 4]
print(nums[:4]) # [0, 1, 2, 3]
print(nums[5:]) # [5, 6, 7, 8, 9]
print(nums[:2]) # [0, 2, 4, 6, 8]
print(nums[-5:-2]) # [5, 6, 7]
```

List Methods

Method	What it does
len(lst)	Number of elements
lst.append(val)	Add val to the end
lst.insert(idx, val)	Insert val at position idx
lst.sort()	Sort ascending / alphabetical (in-place)
lst.reverse()	Flip the order (in-place)

```
nums = [5, 2, 9]
nums.append(7) # [5, 2, 9, 7]
nums.insert(1, 4) # [5, 4, 2, 9, 7]
nums.sort() # [2, 4, 5, 7, 9]
nums.reverse() # [9, 7, 5, 4, 2]
```

Loops & Linear Search on Lists

🌸 Python Fundamentals 🌸 Part 3

Strings • Lists • Tuples • Dictionaries • Sets

```
# Loop through a list
numbers = [10, 20, 30, 40, 50]
for num in numbers:
    print(num)

# Linear Search — check each element one-by-one
numbers = [5, 12, 7, 3, 18, 9]
x = 18
idx = 0
for num in numbers:
    if num == x:
        print(f'{x} found at index {idx}')
        break
    idx += 1
```

■ Tuples

A **tuple** is like a list but **immutable** — once created, its items cannot be changed. Tuples are faster than lists!

```
tup = (10, 20, 30)
print(type(tup)) # <class 'tuple'>
empty_tuple = ()
single_item_tuple = (42,) # trailing comma is REQUIRED for single item
```

Tuple Characteristics

Property	Details
Ordered	Index-based access just like lists
Immutable	Cannot change, add, or remove items after creation
Allows duplicates	Same value can appear more than once
Heterogeneous	Can hold mixed data types

■ *Tuples are faster than lists because of immutability — great for fixed data like coordinates or RGB values.*

Indexing, Slicing & Loops (same as Lists)

```
t = (10, 20, 30, 40)
print(t[0]) # 10
print(t[-1]) # 40
print(t[1:3]) # (20, 30)

# Loop + sum
t = (5, 15, 25)
total = 0
for val in t:
    total += val
print('Sum:', total) # Sum: 45
```

🌸 Python Fundamentals 🌸 Part 3

Strings • Lists • Tuples • Dictionaries • Sets

Tuple Methods

Method	What it returns
t.index(val)	Index of the FIRST occurrence of val
t.count(val)	Total number of times val appears

```
t = (1, 2, 2, 3, 5)
print(t.index(2)) # 1 (first occurrence at index 1)
print(t.count(2)) # 2 (appears twice)
```

■ Dictionaries

A **dictionary** stores data as **key-value pairs**. Think of it like a real dictionary — look up a word (key) to get its meaning (value).

```
my_dict = {
    "name": "Shradha",
    "age" : 30,
    "city": "Delhi"
}
```

Dictionary Characteristics

Rule	Details
Keys must be unique	No two entries can share the same key
Keys must be immutable	Strings, numbers, tuples → OK as keys
Values can be anything	Lists, dicts, numbers, strings ...
Mutable	Add, remove, or update entries freely
Preserves order	Insertion order kept in Python 3.7+

Accessing Values

```
student = {"name": "Bob", "age": 20}
print(student["name"]) # Bob
print(student["age"]) # 20
```

Dictionary Methods

Method	What it does
d.keys()	Returns all keys
d.values()	Returns all values
d.items()	Returns (key, value) pairs as tuples
d.get(key)	Safe access — returns None if key missing (no error!)
d.update({...})	Add or overwrite entries from another dict

🌸 Python Fundamentals 🌸 Part 3

Strings • Lists • Tuples • Dictionaries • Sets

```
d = {"name": "Shradha", "subjects": ["math", "science"], "cgpa": 9.5}

print(d.keys()) # dict_keys(['name', 'subjects', 'cgpa'])
print(d.values()) # dict_values([...])
print(d.items()) # dict_items([...])

print(d.get("cgpa")) # 9.5
print(d.get("phone")) # None (no error!)

d.update({"city": "Delhi"})
print(d) # city key added
```

Looping Through a Dictionary

```
for key, value in d.items():
    print(key, ': ', value)
```

■ Sets

A **set** is an **unordered collection of unique elements**. Duplicates are automatically removed!

```
my_set = {1, 2, 2, 2, 3}
print(my_set) # {1, 2, 3} – duplicates gone!
print(len(my_set)) # 3
empty_set = set() # NOT {} – that makes an empty dict!
```

Set Characteristics

Property	Details
Unique elements only	Duplicates are silently removed
Unordered	No index, no slicing
Mutable	Can add / remove elements
Elements are immutable	Elements must be strings, numbers, or tuples

■ *Use sets when you need uniqueness or maths-style operations (union, intersection).*

Set Methods

Method	What it does
s.add(val)	Add an element
s.remove(val)	Remove val — raises error if not found
s.pop()	Remove & return a random element
s.clear()	Remove all elements → empty set
A.union(B)	All unique elements from both A and B ($A \cup B$)

🌸 Python Fundamentals 🌸 Part 3

Strings • Lists • Tuples • Dictionaries • Sets

A.intersection(B)	Only elements common to both A and B ($A \cap B$)
-------------------	---

```
s = {10, 20, 30}
s.add(40) # {10, 20, 30, 40}
s.remove(10) # {20, 30, 40}
print(s.pop()) # removes a random element
s.clear() # set() – empty set

# Union & Intersection
A = {1, 2, 3}
B = {3, 4, 5}
print(A.union(B)) # {1, 2, 3, 4, 5}
print(A.intersection(B)) # {3}
```

■ Mega Cheat Sheet — All 5 Data Types

Type	Syntax	Ordered?	Mutable?	Duplicates?	Key feature
String	"hi"	■ Yes	■ No	■ Yes	Immutable text, sliceable
List	[1,2]	■ Yes	■ Yes	■ Yes	General-purpose sequence
Tuple	(1,2)	■ Yes	■ No	■ Yes	Faster, fixed data
Dictionary	{k:v}	■ Yes*	■ Yes	Keys: ■	Key-value lookup
Set	{1,2}	■ No	■ Yes	■ No	Unique elements only

Slicing Quick Reference (Strings, Lists & Tuples)

Slice	Result
s[i]	Element at index i (negative = from end)
s[a:b]	Elements from index a up to (not including) b
s[a:]	From a to end
s[:b]	From start to b (exclusive)
s[::n]	Every n-th element
s[::-1]	Reversed sequence!

Common Methods Quick-Ref

Data type	Must-know methods
String	len() • s[i] • s[a:b:c] • f-strings • .format()
List	len() • .append() • .insert() • .sort() • .reverse()
Tuple	.index(val) • .count(val)
Dictionary	.keys() • .values() • .items() • .get() • .update()
Set	.add() • .remove() • .pop() • .union() • .intersection()

🌸 Python Fundamentals 🌸 Part 3

Strings • Lists • Tuples • Dictionaries • Sets

■ You've totally got this! Data structures are just fancy ways to organise information ■